

DATA Transformation

This class handles **data preprocessing** (missing values, encoding, scaling) for a machine learning pipeline, and saves the preprocessing steps as a .pkl file.

Key Components:

1. DataTransformationConfig

- Stores the file path where the **preprocessing object** (ColumnTransformer) will be saved.

```
preprocessor_obj_file_path = 'artifacts/preprocessor.pkl'
```

2. get_data_transformer_object()

- Builds a **preprocessing pipeline**:
 - For **numerical features** (writing_score, reading_score):
 - Fill missing values with median
 - Scale using StandardScaler
 - For **categorical features**:
 - Fill missing values with most frequent value
 - Apply OneHotEncoder
 - Scale using StandardScaler(with_mean=False)
- Combines both pipelines using ColumnTransformer.

3. initiate_data_transformation(train_path, test_path)

- Reads train and test CSV files
- Gets the preprocessing pipeline
- Splits data into input and target features (math_score is the target)
- Applies transformation:
 - fit_transform() on train data
 - transform() on test data

- Combines input and target arrays
- **Saves the preprocessor** object as a .pkl file for reuse

Returns:

- Transformed training data
- Transformed test data
- Path to the saved preprocessor object

Line by line explanation:

Imports

```
import sys
from dataclasses import dataclass

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from src.exception import CustomException
from src.logger import logging
import os

from src.utils import save_object
```

- **Standard libraries:**
 - **sys**: Used to access system-specific parameters (mainly for error handling).
 - **os**: Used for file path management.
- **Data handling:**
 - **numpy, pandas**: For numerical and tabular data processing.
- **Scikit-learn modules:**
 - **SimpleImputer**: Fills in missing values.

- **StandardScaler**: Standardizes numerical features.
- **OneHotEncoder**: Encodes categorical features.
- **Pipeline**: Chains preprocessing steps.
- **ColumnTransformer**: Applies different pipelines to different columns.
- **Project-specific modules**:
 - **CustomException**: A custom class for error handling.
 - **logging**: Logs messages throughout the pipeline.
 - **save_object**: A utility function that likely saves objects using pickle.

Configuration Class

@dataclass

class DataTransformationConfig:

```
preprocessor_obj_file_path = os.path.join('artifacts', "preprocessor.pkl")
```

- Uses **@dataclass** to define configuration for the data transformation stage.
- **Stores the path** where the preprocessor object will be saved as a pickle file.

DataTransformation Class

class DataTransformation:

```
def __init__(self):
```

```
self.data_transformation_config = DataTransformationConfig()
```

- Initializes the class by creating an instance of DataTransformationConfig.

get_data_transformer_object Method

```
def get_data_transformer_object(self):
```

```
"""
```

```
This function is responsible for data transformation
```

```
"""
```

- This method **builds and returns** the full preprocessing pipeline.

1. Define Column Types

```
numerical_columns = ["writing_score", "reading_score"]
categorical_columns = [
    "gender",
    "race_ethnicity",
    "parental_level_of_education",
    "lunch",
    "test_preparation_course",
]
```

- Specifies which columns are numerical and which are categorical.

2. Numerical Pipeline

```
num_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]
)
```

- Fills missing values with median.
- Scales numerical data to mean=0 and std=1.

3. Categorical Pipeline

```
cat_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("one_hot_encoder", OneHotEncoder()),
        ("scaler", StandardScaler(with_mean=False))
    ]
)
```

- Fills missing values with the most frequent category.
- Applies one-hot encoding.
- Scales encoded values (without subtracting the mean).

4. Logging

```
logging.info(f"Categorical columns: {categorical_columns}")
logging.info(f"Numerical columns: {numerical_columns}")
```

- Logs the column names for traceability.

5. Combine Pipelines

```
preprocessor = ColumnTransformer(  
    [  
        ("num_pipeline", num_pipeline, numerical_columns),  
        ("cat_pipelines", cat_pipeline, categorical_columns)  
    ]  
)
```

- Combines numerical and categorical pipelines using ColumnTransformer.

6. Return Preprocessor

```
return preprocessor
```

- Returns the complete transformer object.

7. Exception Handling

```
except Exception as e:  
    raise CustomException(e, sys)
```

- Catches and raises any exception using the custom handler.

initiate_data_transformation Method

```
def initiate_data_transformation(self, train_path, test_path):
```

- Accepts paths to the train and test datasets.

1. Read Data

```
train_df = pd.read_csv(train_path)
```

```
test_df = pd.read_csv(test_path)
```

```
logging.info("Read train and test data completed")
```

- Loads training and testing data.
- Logs the successful read.

2. Get Preprocessing Object

```
logging.info("Obtaining preprocessing object")  
preprocessing_obj = self.get_data_transformer_object()
```

- Retrieves the full preprocessing pipeline.

3. Setup Target and Feature Columns

```
target_column_name = "math_score"  
numerical_columns = ["writing_score", "reading_score"]
```

- Specifies the target (label) and numerical columns again.

4. Split Features and Target

```
input_feature_train_df = train_df.drop(columns=[target_column_name], axis=1)  
target_feature_train_df = train_df[target_column_name]
```

```
input_feature_test_df = test_df.drop(columns=[target_column_name], axis=1)  
target_feature_test_df = test_df[target_column_name]
```

- Separates features and target in both train and test datasets.

5. Apply Transformation

```
logging.info("Applying preprocessing object on training dataframe and testing dataframe.")
```

```
input_feature_train_arr = preprocessing_obj.fit_transform(input_feature_train_df)  
input_feature_test_arr = preprocessing_obj.transform(input_feature_test_df)
```

- Fits and transforms training features.
- Transforms testing features using the same pipeline.

6. Combine Features and Target

```
train_arr = np.c_[input_feature_train_arr, np.array(target_feature_train_df)]  
test_arr = np.c_[input_feature_test_arr, np.array(target_feature_test_df)]
```

- Combines features and targets back into final numpy arrays (for model input).

7. Save Preprocessing Object

```
logging.info(f"Saved preprocessing object.")
```

```
save_object(  
    file_path=self.data_transformation_config.preprocessor_obj_file_path,  
    obj=preprocessing_obj  
)
```

- Saves the preprocessor object as a .pkl file for future reuse.

8. Return Final Outputs

```
return (  
    train_arr,  
    test_arr,  
    self.data_transformation_config.preprocessor_obj_file_path,  
)
```

- Returns the final transformed arrays and the file path of the saved preprocessor.

9. Exception Handling

```
except Exception as e:
```

```
    raise CustomException(e, sys)
```

- Catches and raises errors with full traceback for debugging.

Summary

This class:

- Builds a data preprocessing pipeline
- Handles missing values, encoding, and scaling
- Applies transformation to train and test datasets
- Saves the pipeline for use in prediction or inference